

DS310 Lab 2 – Introduction to SQL with SQLite

In this lab you will write and run SQL queries against a SQLite database using Python, inside a Jupyter notebook in VS Code.

Setting Up: Python, VS Code, and the Database

- Install Visual Studio Code (code.visualstudio.com) if you don't already have it. Then install the Python extension and the Jupyter extension from the Extensions view (Ctrl+Shift+X, or Cmd+Shift+X on Mac).
- Make sure Python 3 is installed on your machine. SQLite support (the `sqlite3` module) is already built into Python, so there's nothing extra to install for that. You will also use `pandas`, so open a terminal (Terminal > New Terminal in VS Code) and run:

```
pip install pandas
```

- Download the database file `university.sqlite` from Piazza, along with the notebook template `DS310-Lab2-F26-Template.ipynb`. Put both files in the same folder.
- In VS Code, choose File > Open Folder... and open that folder, then open `DS310-Lab2-F26-Template.ipynb`.
- The first time you run a cell (Shift+Enter, or the ▶ button to its left), VS Code will prompt you to select a kernel — choose your Python 3 installation.
- Run the setup cell at the top of the notebook. It imports `sqlite3` and `pandas` and opens a connection to the database:

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("university.sqlite")
```

- Keep that `conn` connection open — you'll reuse it for every query in this lab. There's no need to reconnect for each one.

Task 1: Explore the database with Python

In this task, you will get familiar with the university database and with running SQL queries from Python. Note that the tables have the same column names as the university database from the lecture notes, but the actual rows are different. In particular:

- We have included some additional data to allow for more interesting queries and results.
- We use the full names of the departments in the `Department` and `MajorsIn` tables. In the lecture notes, the department names are abbreviated.
- In a new cell, view the contents of a table by running a query like the following. Because `conn` is a live SQLite connection, `pandas` returns the result as a nicely formatted table (a `DataFrame`):

```
pd.read_sql_query("SELECT * FROM Course", conn)
```

- Explore the other tables the same way (Student, Department, Room, MajorsIn, Enrolled) by changing the table name.
- Now, in a new cell, run the following query:

```
pd.read_sql_query("""
SELECT name, credit_status
FROM Student, Enrolled
WHERE id = student_id
AND course_name = 'CS 640';
""", conn)
```

- What results do you see? What do they represent?
- Edit the previous query to remove the first condition in the WHERE clause, then rerun the cell:

```
pd.read_sql_query("""
SELECT name, credit_status
FROM Student, Enrolled
WHERE course_name = 'CS 640';
""", conn)
```

- How does removing that condition affect the results?
- Enter and run the following query in a new cell:

```
pd.read_sql_query("""
SELECT name
FROM Course
WHERE room_id IS NULL;
""", conn)
```

- What does this query do? Does the following query do the same thing as the previous one?

```
pd.read_sql_query("""
SELECT name
FROM Course
WHERE room_id = NULL;
""", conn)
```

- Why or why not?

Task 2: Understand and write SQL queries

In this task, you will continue to work with the university database from the previous task. Here is its schema:

Schema of the database

Student(id, name)

Department(name, office)

Room(id, name, capacity)

Course(name, start_time, end_time, room_id)

MajorsIn(student_id, dept_name)

Enrolled(student_id, course_name, credit_status)

When constructing each query, we recommend that you follow the rules of thumb given in the lecture notes.

Feel free to work with another student on these problems, and to ask the TA for help as needed. For each problem below, write your query as a Python string and run it with `pd.read_sql_query(query, conn)` in a new cell of the notebook.

1. Write a query that finds the names and capacities of all rooms in the CAS and CGS buildings. Test your query and revise it as needed.
2. Write a query that finds the number of students who are enrolled in a course for undergraduate ('ugrad') credit. Test your query. The correct result is 8. If you don't get this result, revise it as needed.
3. Write a query that finds the earliest (i.e., smallest) start time of any DS course.
4. Modify your previous query to find the name and start time of the DS course with the earliest start time. To adhere to standard SQL, you will need to use a subquery.

Important: you should not do something like the following:

```
SELECT name, MIN(start_time)
...
```

Although this works in some versions of SQL, e.g., SQLite, it is not allowed in standard SQL, because the SELECT clause combines a regular column (name) with an aggregate function (MIN(start_time)).

5. Write a query that finds, for each type of credit status, the number of students who are enrolled in a course with that credit status. Note: for this query, you are allowed to combine a regular column with an aggregate function. Why is it allowed in this case?
6. Write a query that finds the names of all students majoring in data science. Hint: don't forget to include an appropriate join condition.
7. Write a query that finds the names of all students who are not enrolled in DS 310.

Task 3: Understand and write relational-algebra queries

On paper, write a relational-algebra query for each of the following problems from Task 2:

1. Problem 6: (names of students majoring in data science)
2. Problem 7 (names of students not enrolled in DS 310). Hint: take advantage of the set difference operator (-). To make this query easier to write, you may assume that all student names are unique.

FYI: Alternative Way to Test Your Queries — DB Browser for SQLite

Everything in this lab can be done in Python. If you'd also like an optional graphical way to browse the database and test queries — outside your notebook — you can install DB Browser for SQLite and use it alongside the Python approach above. This section is entirely optional and not required for completing the lab.

- Go to sqlitebrowser.org/dl. Find the appropriate download link for your machine/operating system and download and install that version of DB Browser for SQLite.
- Launch DB Browser for SQLite.
- Click the Open Database button and open the same university.sqlite file you downloaded earlier.
- Click on the Browse Data tab to explore the contents of the tables, choosing a table name from the drop-down menu.
- Select the Execute SQL tab, which is where you can type and run SQL queries directly, without Python. You can paste in the SQL from inside any of the `pd.read_sql_query("""...""", conn)` calls in this lab.
- To run a query, do any of the following: press F5, press Ctrl-R, or click the small button with a triangular shape that looks like the Play button of a music player.